android15-virtual-ab

Android 15 Virtual A/B with Compression — Device-Side Core Research Note

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Primary-source research on Android 15 Virtual A/B with compression (VABc) complete. Covers snapshot/COW internals (dm-snapshot deprecation, dm-user, snapuserd), dynamic partitions (`super`), compression methods (none/gz/lz4/zstd + XOR) and storage tradeoffs, the post-OTA merge phase and first-boot flow, failure handling / cancel semantics, and RK3588 / Orange Pi 5 Max considerations. Conclusion: Virtual A/B compressed (lz4 default, zstd optional) is the device-side mechanism that delivers the "no-corruption" guarantee through atomic slot-switch + power-fail-resumable userspace merge; Helix must build payloads correctly and orchestrate, not re-implement the apply/merge path.
Issues	Several quantitative figures (VABc $_{45\%/}$55% snapshot reduction, XOR 25–40% reduction, sizing example outputs, `FinalOTASnapshotSize = FinalDessertSize × 0.7`) are taken from current AOSP docs prose and are version-sensitive; re-confirm against the exact Android 15/16 docs revision before quoting in contracts — marked UNVERIFIED where exactness matters. RK3588 / Orange Pi 5 Max Virtual A/B status is **not** documented by AOSP and

Field	Value
	depends on the Rockchip BSP / vendor build config; treated as UNVERIFIED and flagged as a required hardware-validation task.
Fixed	N/A (first revision)
Continuation	Follow-up should: (1) pin exact Android 15 (`vabc_features.mk`) defaults and confirm `PRODUCT_VIRTUAL_AB_COMPRESSION_METHOD` + `PRODUCT_VIRTUAL_AB_COMPRESSION_FACTOR` defaults; (2) empirically benchmark lz4 vs zstd merge time / CPU / storage on the target RK3588 board; (3) confirm whether the chosen RK3588 Android 15 BSP ships VABc-enabled, dynamic partitions, and `dm-user` kernel module; (4) define Helix telemetry for `MergeStatus` transitions and merge-resume failures; (5) document the `update_engine` cancel contract and Helix's safe-cancel window.

Table of Contents

1. Scope & Question

Helix OTA targets **Android 15 first**, using **native A/B + Virtual A/B** on device with a **custom Go control plane** as the server. This note is the **device-side core**: it explains exactly how a modern Android 15 device applies an update without risking a corrupt/bricked state, so that Helix can build the right artifacts and orchestrate safely. It answers:

- How do **dynamic partitions** (`super`) and **Virtual A/B** work together?

- What are the **snapshot / COW** internals (dm-snapshot, dm-user, snapuserd)?
- What **compression** options exist (none/gz/lz4/zstd, plus XOR) and what are the **storage tradeoffs**?
- What happens in the **merge phase** and **post-OTA first boot**?
- How does **failure handling / cancel / rollback** preserve the no-corruption guarantee?
- What is the situation on **RK3588 / Orange Pi 5 Max**?

This is the underpinning for the claim that the **on-device `update_engine` + Virtual A/B path is the trustworthy atomic core**, and Helix should wrap it (see `aosp-update-engine.md` and `adr-0001-wrapped-engine.md`).

---

2. Executive Summary

- **Virtual A/B (VAB)** is Android's seamless-update mechanism that gives you A/B safety (two slots, atomic switch, automatic rollback) **without** physically duplicating every dynamic partition. Instead of a full B copy, the update is written to a **Copy-on-Write (COW)** area as a **snapshot**, then **merged** into the base after a confirmed-good boot. It is driven by the same `update_engine` daemon as classic A/B. (AOSP Virtual A/B overview)
- **Virtual A/B Compression (VABc)** compresses the COW block data. AOSP states the COW format supports four operations — **Copy, Replace (compressed), Zero, XOR** — and that compression cuts snapshot size by roughly **~45% on a full OTA and ~55% on an incremental OTA** (UNVERIFIED exact figures; from AOSP prose). (AOSP Virtual A/B overview)
- **COW format / device-mapper evolution:** Android 11 used the **kernel COW format** (no compression); Android 12 added an **Android-specific COW format** with compression (still translated to kernel COW for merge); **Android 13+ removed reliance on kernel COW *and* `dm-snapshot`**, moving the entire snapshot read/write/merge into **userspace** via the **`dm-user`** kernel shim + **`snapuserd`** daemon. Android 15 inherits the Android 13+ userspace model. (AOSP Virtual A/B overview)
- **Compression methods** for VABc: **`lz4` (default), `zstd`, `none`**, configured via `PRODUCT_VIRTUAL_AB_COMPRESSION_METHOD`; a `PRODUCT_VIRTUAL_AB_COMPRESSION_FACTOR` (4k…256k, default 64k) controls the max compression window. **XOR compression** (Android 13+) is a separate, complementary feature. (AOSP Implement Virtual A/B) `gz`/gzip appears in the historical COW format; **lz4 and zstd are the practical Android 13+ choices**. (gz status for Android 15 marked UNVERIFIED.)
- **Storage tradeoff:** VAB shrinks `super` to roughly half the classic-A/B allocation (no resident B slots), but the COW snapshot needs transient space during the update — placed in `super` if it fits, otherwise spilling to **`/data`**. AOSP's comparison table: classic A/B ≈ 9 GB resident `super`; VAB ≈ 4.5 GB `super` + ~3.8 GB transient on `/data`; **VAB compressed ≈ 4.5 GB `super` + ~2.1 GB transient on `/data`**. (UNVERIFIED exact numbers; illustrative from AOSP.) (AOSP Virtual A/B overview, Size the super partition)
- **Lifecycle:** `update_engine` writes the inactive slot's snapshots (COW) → bootloader switches active slot → device boots new slot with `/system` mounted **through `dm-verity` over `dm-user`** (I/O served by `snapuserd`) → on `boot completed`, `update_engine` calls `ScheduleWaitMarkBootSuccessful()` / `WaitForMergeOrSchedule()`, marks boot successful, then runs the **merge** (snapshot collapsed into base; `dm-verity` collapses back onto `dm-linear`, `dm-`

`user` removed). Merge is **resumable across reboots** and **power-fail safe**. (AOSP Virtual A/B overview, Implement Virtual A/B - patches)

- **Failure handling = the no-corruption guarantee:** if the new slot fails to boot, the bootloader rolls back to the old slot (snapshot still discardable). Merge interruptions don't lose data; merge resumes after reboot. The dangerous window is **after merge has started but before completion**: rebooting to the old slot then is unsafe and AOSP has explicit CLs to prevent it. (AOSP Implement Virtual A/B - patches)
- **RK3588 / Orange Pi 5 Max: UNVERIFIED.** AOSP documents the mechanism, not specific SoCs. Whether a given RK3588 Android 15 build ships **dynamic partitions + VAB + VABc + `dm-user`** depends entirely on the **Rockchip BSP and the integrator's board config**; many Rockchip Android images historically ship a Rockchip-specific OTA flow and may not enable dynamic partitions/VAB by default. This must be validated on the actual target image before Helix can rely on the native VAB path.
- **Conclusion:** the Virtual A/B compressed path is the right device-side core for Helix's no-corruption guarantee — it provides atomic slot switch, automatic rollback, and a power-fail-resumable userspace merge. Helix's job is to (a) **produce a correct payload** that targets this mechanism, (b) **drive `applyPayload` and observe merge status**, and (c) **never** introduce a code path that reboots to the old slot mid-merge.

---

3. Dynamic Partitions & the `super` Partition

Dynamic partitions (Android 10+) are a userspace partitioning system that lets the OS create/resize/destroy logical partitions (e.g., `system`, `vendor`, `product`, `system_ext`) at OTA time without a fixed physical partition table. They all live inside one physical container partition called **`super`** (`/dev/block/by-name/super`). (AOSP OTA for A/B devices with dynamic partitions)

Inside `super`, the normal mount stack for a read-only system image is:

1. Physical `super` partition.
2. **`dm-linear`** — maps the logical partition's extents within `super`.
3. **`dm-verity`** — cryptographic block-level integrity over the partition (this is part of what makes corruption *detectable* and rollback-triggering).

Dynamic partitions are the prerequisite for Virtual A/B: because logical partitions can be re-described cheaply, VAB can keep a single base copy and overlay a snapshot rather than reserving a whole second physical slot. (AOSP Virtual A/B overview)

---

4. Virtual A/B vs Classic A/B

Aspect	Classic A/B	Virtual A/B (compressed)
Slots	Two **physical** copies of each updatable partition	One base copy in `super`; second "slot" is a **snapshot/COW** overlay

Aspect	Classic A/B	Virtual A/B (compressed)
Resident storage	~2× partition size always resident	~1× resident; COW is **transient**, reclaimed after merge
Boot partitions (`boot`, etc.)	Duplicated	Still **duplicated** (small); only dynamic partitions are snapshotted
Update applied by	`update_engine`	`update_engine` (same daemon)
Rollback	Bootloader slot switch	Bootloader slot switch (discard snapshot)
Extra step	None	**Merge phase** after confirmed boot
Compression	N/A	none / lz4 / zstd (+ XOR)

Key point: VAB **keeps all the A/B safety properties** (atomic switch, automatic rollback, "seamless" background install while the device is usable) while trading a small amount of **post-boot merge time** and **transient COW space** for a much smaller resident footprint. (AOSP Virtual A/B overview; see also `aosp-update-engine.md`.)

---

5. Snapshot / COW Internals

5.1 COW format and operations

The Android COW format supports four block operations (AOSP Virtual A/B overview):

- **Copy** — replace block X with block Y read from the **base** device (intra-partition move; no new data stored).
- **Replace** — replace block X with a **compressed** block Y stored in the snapshot (the bulk of "new" data).
- **Zero** — fill block X with zeros (cheap; stores no payload).
- **XOR** (Android 13+) — store the **XOR difference** between the old and new block. When only a few bytes change, the XOR is mostly zeros and compresses extremely well, so the snapshot stores far less than a full 4K block.

5.2 COW format / kernel evolution (why Android 13+ matters for Android 15)

- **Android 11: kernel COW format** via `dm-snapshot`; **no compression**.
- **Android 12: Android-specific COW format** that supports compression, but it had to be **translated to the kernel COW format** before the kernel `dm-snapshot` merge.
- **Android 13+ (and thus Android 15):** removed reliance on **both** the kernel COW format **and** `dm-snapshot`. The snapshot is now **read, written, and merged entirely in userspace** by `snapuserd`, with the kernel only providing the **`dm-user`** passthrough block device.

(AOSP Virtual A/B overview)

This is significant for Helix: on Android 15 the merge is a **userspace** operation, so its progress, pausing, and resumption are observable/controllable through `update_engine` / `snapuserd` rather than buried in kernel `dm-snapshot`.

---

6. The Compressed Device-Mapper Stack (dm-user / snapuserd)

With **compressed** snapshots the read path for `/system` becomes (AOSP Virtual A/B overview):

```
super (physical)
  └─ dm-linear  (system_base: extents within super)
       └─ dm-user  (routes block I/O to a userspace daemon)
            └─ dm-verity (integrity verification)
                 └─ /system mount
```

- **`dm-user`** is a kernel module that creates a control device at **`/dev/dm-user/<control-name>`**. The kernel forwards block read/write requests to a **userspace** process that polls this device. AOSP notes `dm-user` is a **non-upstream** kernel interface that Google reserves the right to modify. (Kernel config: `CONFIG_DM_USER=m` or `y`; if modular, load it in the first-stage ramdisk via `BOARD_GENERIC_RAMDISK_KERNEL_MODULES_LOAD`.) (AOSP Implement Virtual A/B)
- **`snapuserd`** is the userspace daemon that **implements** the COW: during OTA install it **writes compressed** new block data into the snapshot; during normal operation and during merge it **serves reads** (decompressing on the fly) and **performs the merge**. All snapshot I/O goes through it. (AOSP Virtual A/B overview)

6.1 The init / snapuserd handoff (why SELinux ordering matters)

Booting with compressed snapshots requires a careful, synchronized **init ⁉ snapuserd transition** to avoid an I/O deadlock when SELinux policy is loaded (the daemon serving the pages that init needs to read cannot itself be blocked). The AOSP-documented sequence (AOSP Virtual A/B overview):

1. First-stage `init` launches the **ramdisk** `snapuserd`, saves its file descriptor to the environment.
2. Root switches to the system partition; **system `init`** runs.
3. System `init` reads the combined sepolicy and `mlock()`s the ext4-backed pages it needs.
4. Device-mapper snapshot tables are deactivated; the ramdisk `snapuserd` is **stopped** (preventing the IO deadlock).
5. Using the preserved fd, `init` **relaunches `snapuserd`** with the correct SELinux context; tables are reactivated.
6. `init` calls `munlockall()`; normal I/O resumes.

In Android 13, `snapuserd` moved from the **vendor** ramdisk to the **generic** ramdisk, which affects recovery and `fastbootd` boot sequences. (AOSP Implement Virtual A/B - patches)

---

7. Compression: none / gz / lz4 / zstd + XOR

7.1 Methods and configuration

Per AOSP Implement Virtual A/B, VABc compression methods are:

- **`lz4`** — default for compressed VAB; very fast compress/decompress, modest ratio. Good for merge speed and low CPU.
- **`zstd`** — better ratio than lz4 at a configurable speed/ratio level; decompression speed stays roughly constant across levels. Better for minimizing snapshot/ transfer size when CPU/merge-time budget allows.
- **`none`** — uncompressed (largest transient footprint).

Android 13+ enable example:

```
$(call inherit-product, $(SRC_TARGET_DIR)/product/
        generic_ramdisk.mk)
$(call inherit-product, $(SRC_TARGET_DIR)/product/virtual_ab_ota/
        vabc_features.mk)

PRODUCT_VIRTUAL_AB_COMPRESSION_METHOD := lz4      # or zstd / none
PRODUCT_VIRTUAL_AB_COMPRESSION_FACTOR := 65536    # 64k window
```

- **`PRODUCT_VIRTUAL_AB_COMPRESSION_FACTOR`** = max compressible window. Supported: **4k, 8k, 16k, 32k, 64k, 128k, 256k** (default **64k**). Larger windows can improve ratio at some CPU/memory cost. (AOSP Implement Virtual A/B)
- **Compression level** is algorithm-specific (zstd exposes levels trading speed vs ratio). (AOSP Implement Virtual A/B)

gz/gzip: present in the historical Android COW format description; for Android 13+/15 the documented, supported configuration values are **lz4 / zstd / none**. Treat **gz as legacy** for Android 15 unless the target build explicitly exposes it — **UNVERIFIED** for Android 15.

7.2 XOR compression (Android 13+)

A separate, complementary feature: stores **XOR-compressed differential bytes** between old and new blocks. AOSP states it reduces snapshot size by roughly **25–40%** when only a few bytes per block change (UNVERIFIED exact figures). It is most valuable for **incremental/delta** OTAs where many blocks change slightly. Note AOSP wording is inconsistent across pages on whether XOR is "enabled by default" vs "disabled by default" in a given release — **UNVERIFIED**; confirm for the exact Android 15 build. (AOSP Virtual A/B overview, AOSP Implement Virtual A/B)

7.3 Algorithm tradeoff summary (for Helix tuning)

Method	Compress speed	Decompress speed	Ratio	Merge/ CPU cost	When to pick
none	n/a	fastest	none	lowest CPU,	

Method	Compress speed	Decompress speed	Ratio	Merge/ CPU cost	When to pick
				largest space	abundant storage, weak CPU
`lz4` (default)	very fast	very fast	low–moderate	low	balanced default; constrained CPU like an SBC
`zstd`	fast (level-tunable)	fast & stable	higher	higher than lz4	minimize transient footprint / `/data` pressure

Quantitative algorithm comparisons for ratio/speed are well established generally (zstd > gzip ratio at higher speed; lz4 fastest) per general benchmarks (lzbench), but **AOSP does not publish merge-time/CPU numbers per method for VAB** — Helix must **benchmark on the actual RK3588 target**. (UNVERIFIED for the specific board.)

---

8. Storage Tradeoffs & `super` Sizing

VAB lets you **shrink `super`** (no resident B slots) to give more room to `/data`, but the COW snapshot needs **transient** space during an update; if it doesn't fit in `super` it spills to **`/data`**. If users lack free `/data`, **update success rates drop**. (AOSP Size the super partition)

AOSP sizing formulas (AOSP Size the super partition) — figures **UNVERIFIED** as exact, version-sensitive:

VAB without compression:

```
FinalDessertSize = FactorySize + (FactorySize × ExpectedGrowth)
# relying on /data:
Super = Max(FinalDessertSize, FinalDessertSize × 2 −
AllowedUserdataUse)
# never touch /data:
Super = FinalDessertSize × 2
```

VAB with compression:

```
FinalOTASnapshotSize = FinalDessertSize × 0.7
# relying on /data:
Super = Max(FinalDessertSize, FinalDessertSize +
FinalOTASnapshotSize − AllowedUserdataUse)
# never touch /data:
Super = FinalDessertSize + FinalOTASnapshotSize
```

Worked example (4 GB factory, 50% growth) from AOSP: - Uncompressed, relying on 1 GB free: final 6 GB → `super` **11 GB**. - Compressed, relying on 1 GB free: final 6 GB, snapshot 4.2 GB → `super` **9.2 GB**.

Illustrative footprint comparison (AOSP overview table; UNVERIFIED exact): classic A/B ≈ 9 GB resident `super`; VAB ≈ 4.5 GB `super` + ~3.8 GB transient on `/data`; **VAB compressed ≈ 4.5 GB `super` + ~2.1 GB transient on `/data`**. The transient space is reclaimed after merge/reboot. (AOSP Virtual A/B overview)

Helix takeaway: on storage-constrained SBC-class devices, **compression (lz4 minimum, zstd if CPU allows) materially raises update success rates** by reducing COW spill into `/data`. Helix's control plane should track free `/data` and treat low-space devices as elevated-risk for a given rollout.

---

9. The Update Lifecycle: Write → Reboot → Merge → Complete

1. **Install / write phase (device online, slot A active).** `update_engine` applies `payload.bin` to the **inactive** slot, writing new block data **into the COW snapshot** (via `snapuserd`, compressed). The active slot keeps running; the update is "seamless." (AOSP Virtual A/B overview)
2. **Slot switch.** When the payload is fully written and verified, the bootloader is told to boot the **new** slot next (boot_control HAL). The COW/snapshot is what makes the new slot's view of `/system` etc.
3. **First boot of new slot.** The system boots with `/system` mounted via **`dm-verity` over `dm-user`**; all reads are served by `snapuserd` (base block, or decompressed snapshot block, or XOR-reconstructed block). dm-verity still verifies integrity, so a corrupt block is **detected**, not silently used.
4. **Mark boot successful + schedule merge.** On boot completion (Android 11+), `update_engine` calls **`ScheduleWaitMarkBootSuccessful()`** and **`WaitForMergeOrSchedule()`**, marks the slot successful (so the bootloader won't fall back), and starts the **merge**. (AOSP Virtual A/B overview, Implement Virtual A/B - patches)
5. **Merge phase (userspace).** `snapuserd` collapses the COW snapshot into the base device. AOSP describes the teardown: after merge completes the framework **collapses `dm-verity` back onto `dm-linear`** and **removes the `dm-user`** layer. Merge "usually takes a few minutes," runs in the background, and is **resumable across reboots without data loss**. After merge, the update is **complete**. (AOSP Virtual A/B overview)
6. **MergeStatus tracking.** The boot_control HAL exposes a `MergeStatus` enum: **`NONE`, `UNKNOWN`, `SNAPSHOTTED`, `MERGING`, `CANCELLED`** — the canonical state machine Helix should surface in telemetry. (AOSP A/B docs / boot_control HAL)

---

10. Failure Handling, Cancel & Rollback

This section is the heart of the **no-corruption guarantee**.

- **Failed first boot → automatic rollback.** If the new slot fails to boot (e.g., it never marks boot successful within the bootloader's retry budget), the bootloader falls back to the **old slot**. Because the merge has **not** happened yet, the base is intact and the snapshot is simply discarded. No corruption. (AOSP Virtual A/B overview)
- **dm-verity integrity.** Reads through the new slot are verity-checked; corruption is **detected** rather than consumed, which is what allows the system to fail safely toward rollback.
- **Power-fail during install.** The slot switch is the atomic commit point; a crash before it leaves the device booting the old slot normally. The COW snapshot is just discarded.
- **Power-fail during merge.** Merge is **resumable**: the COW/merge state is persisted, and on reboot `snapuserd` resumes where it left off. No data loss. (AOSP Virtual A/B overview)
- **The dangerous window — reboot to old slot *after merge has started*.** AOSP explicitly warns that if merging has begun and the device then reboots to the **old** slot, it can become inoperable. There are specific CLs/patches to prevent premature merge initiation and to enforce correct state tracking across reboots. **Implication for Helix: never trigger a slot revert once `MergeStatus == MERGING`.** (AOSP Implement Virtual A/B - patches)
- **Cancel semantics.** `update_engine_client --cancel` during the merge phase historically caused a **segfault**; the fix added `StopActionInternal` to cancel pending tasks safely and track the pending task ID. Helix should: (a) only cancel during the **install** phase (safe), (b) treat cancel during **merge** as unsupported/forbidden, and (c) verify the target build includes the relevant fixes. (AOSP Implement Virtual A/B - patches)
- **Other documented failure modes** (apply patches before relying on VAB in production): allocation-space verification when sideloading, wild-pointer errors when slot-success marking is delayed, and corrupted `vbmeta` from improper dm-verity configuration. (AOSP Implement Virtual A/B - patches)
- **Recovery / fastbootd.** Because `snapuserd` moved to the generic ramdisk (Android 13+), VAB-aware flows exist in both recovery and `fastbootd`; sideload/recovery OTA must allocate COW space correctly. (AOSP Implement Virtual A/B - patches)

Net: the only state that can brick a VAB device is mishandling the **post-merge-start** window. Everything else (failed boot, power loss pre-commit, power loss mid-merge) is recoverable by design. Helix's safety model should encode "no revert after merge start" as an invariant.

---

11. RK3588 / Orange Pi 5 Max Considerations

Overall status: UNVERIFIED — requires hardware/image validation. AOSP documents the *mechanism*; it does not certify specific SoCs/boards. The following are the concrete things Helix must confirm on the **actual Android 15 image** for the RK3588 / Orange Pi 5 Max target:

1. **Is the build A/B (not A-only)?** Many Rockchip Android factory images historically ship **A-only** with a **Rockchip-proprietary OTA** (e.g., `rkupdate` / `update.img`) rather than AOSP `update_engine` A/B. If the target is A-only, the AOSP Virtual A/B path **does not apply** and the no-corruption story changes substantially. **UNVERIFIED — confirm.**
2. **Are dynamic partitions enabled?** VAB requires `super` / dynamic partitions (`BOARD_SUPER_PARTITION_SIZE`, update groups). Confirm the board config defines `super` and logical partitions. **UNVERIFIED.**
3. **Is VAB enabled, and VABc?** Confirm `PRODUCT_VIRTUAL_AB_OTA` (and `vabc_features.mk` / `PRODUCT_VIRTUAL_AB_COMPRESSION_METHOD`) are inherited in the device makefiles. **UNVERIFIED.**
4. **Kernel support.** RK3588 vendor kernels are typically **5.10-class** (Rockchip BSP). VAB needs `CONFIG_DM_SNAPSHOT`; compressed VAB needs **kernel 4.19+** and **`CONFIG_DM_USER`** (built-in or modular + loaded in first-stage ramdisk). Confirm `dm-user` is present — it is a **non-upstream** module, so it must be carried in the Rockchip kernel. **UNVERIFIED.**
5. **Storage class & sizing.** Orange Pi 5 Max uses eMMC and/or NVMe (RK3588). Compression (lz4) is recommended to limit COW spill into `/data`; pick `super` sizing per §8. eMMC write endurance/throughput affects merge time — **benchmark**. **UNVERIFIED.**
6. **CPU for compression.** RK3588 is an 8-core (4×A76 + 4×A55) SoC — comfortably able to run lz4 and likely zstd at reasonable levels during merge, but **decompression on every `/system` read until merge completes** adds latency on the post-OTA boot. Benchmark lz4 vs zstd merge time and first-boot read latency on the board. **UNVERIFIED.**
7. **Bootloader / AVB.** RK3588 uses U-Boot + Rockchip miniloader/idbloader; A/B slot management and AVB/vbmeta must be wired into `boot_control` for rollback to actually work. Confirm `boot_control` HAL is the AOSP one (not a Rockchip stub). **UNVERIFIED.**

Practical guidance: Helix should treat "native AOSP Virtual A/B available and correct on this RK3588 image" as a **gating prerequisite** that must be empirically proven (apply a test OTA, observe `MergeStatus` transitions, force a failed boot to confirm rollback, force power loss mid-merge to confirm resume). If the stock Rockchip image is A-only/proprietary, a separate ADR is needed on whether to (a) bring up AOSP VAB on the board, or (b) wrap the Rockchip OTA mechanism instead.

Sources for hardware context: Orange Pi 5 / RK3588 user manuals, 7Ji/orangepi5-rkloader (bootloader/GPT layout). These confirm the boards' boot/partition basics but **do not** confirm AOSP VAB enablement.

---

12. Implications for Helix OTA (the no-corruption guarantee)

1. **The atomic core is the device's `update_engine` + Virtual A/B.** Helix must **wrap**, not re-implement, the install/merge path (consistent with `adr-0001-wrapped-engine.md`). The corruption-resistance comes from: atomic slot switch, dm-verity detection, automatic rollback on failed boot, and the resumable userspace merge.
2. **Build payloads that target VABc.** Payload generation (`ota_from_target_files`, see `aosp-update-engine.md`) must produce a payload whose COW operations match the device's enabled compression. Helix's build pipeline should pin the compression method and factor and keep them consistent with the device build.
3. **Default to `lz4`, offer `zstd` per fleet segment.** lz4 minimizes CPU/merge time (good for SBC-class RK3588); zstd reduces transient `/data` pressure on storage-constrained devices. Make this a per-rollout knob driven by device telemetry (free `/data`, SoC class).
4. **Surface `MergeStatus` as first-class telemetry.** Track `SNAPSHOTTED → MERGING → (complete)` and alert on stuck/`CANCELLED`. The control plane should consider an update "done" only after merge completion, not at reboot.
5. **Encode the safety invariant: no revert after `MERGING`.** Helix must never instruct a device to switch back to the old slot once merge has started. Cancel is only safe during install.
6. **Gate on free `/data`.** Low-space devices have lower update success; the control plane should defer or warn, and prefer compression for them.
7. **RK3588 enablement is a prerequisite, not an assumption.** Validate VAB availability on the real image before relying on the native path; otherwise raise an ADR.

---

13. Open Questions / UNVERIFIED Items

- Exact Android 15 defaults for `PRODUCT_VIRTUAL_AB_COMPRESSION_METHOD` and whether XOR is on/off by default. **UNVERIFIED.**
- Exact VABc/XOR space-saving percentages and the AOSP footprint table numbers for the Android 15 docs revision. **UNVERIFIED.**
- `gz`/gzip availability as a selectable VABc method on Android 15 (appears legacy). **UNVERIFIED.**
- Whether the target RK3588 / Orange Pi 5 Max Android 15 image ships A/B + dynamic partitions + VAB + VABc + `dm-user` (vs A-only Rockchip OTA). **UNVERIFIED — top-priority hardware validation.**
- Real lz4-vs-zstd merge time, CPU, and first-boot read-latency numbers on the RK3588 target. **UNVERIFIED — benchmark.**
- Precise `update_engine` cancel contract and which fixes the target build includes. **UNVERIFIED — confirm against build.**

---

14. Sources Consulted

Primary (AOSP, consulted directly): - Virtual A/B overview — source.android.com - Implement Virtual A/B — source.android.com - Implement Virtual A/B - patches — source.android.com - A/B (seamless) system updates — source.android.com - OTA for A/B devices with dynamic partitions — source.android.com - Size the super partition — source.android.com - Reduce OTA size — source.android.com

Source / code references (referenced, not all line-by-line read): - platform/system/update_engine — Git at Google - fs_mgr/libsnapshot/snapshot.cpp — Git at Google - snapuserd dm-snapshot-merge readahead — cs.android.com - platform/external/zstd — Git at Google

Hardware context (RK3588 / Orange Pi 5 — do not confirm VAB enablement): - Orange Pi 5 RK3588S User Manual (PDF) - 7Ji/orangepi5-rkloader — GitHub

Secondary / context: - Android 13's Virtual A/B Mandate — esper.io - lzbench compression benchmark

Internal cross-references: `aosp-update-engine.md`, `adr-0001-wrapped-engine.md`, `adr-0005-delta-updates.md`.

---

15. Confidence

Overall confidence: MEDIUM-HIGH on the Virtual A/B / VABc mechanism, COW internals, dm-user/snapuserd architecture, merge lifecycle, and failure/cancel semantics — these are drawn directly from current AOSP primary documentation and corroborated across multiple AOSP pages.

LOW confidence on (a) exact quantitative figures (compression %s, sizing table outputs, XOR default state) which are version-sensitive prose and flagged UNVERIFIED, and (b) **everything specific to RK3588 / Orange Pi 5 Max**, where AOSP provides no guarantees and the outcome depends entirely on the Rockchip BSP and the integrator's board configuration. The RK3588 VAB enablement question is the single highest-priority item to validate empirically before Helix relies on the native Virtual A/B path for its no-corruption guarantee.